

Rung 5 — Why the loop can't escape (safety that holds forever)

Where you start: Rungs 1–4 bounded the error of *one* computation. But the guarded kernels that started this whole project run a **loop** — read a sensor, decide, drive an output, repeat, thousands of times a second. This rung bounds a system that runs *forever*.

What you'll be able to do: explain why a control loop's state can never leave a safe region — not "we tested it for an hour," but a proof that covers step number ten billion.

1. The bench

The `pid_dual_target` kernel runs a proven controller around a real analog plant (an RC circuit, and a nastier diode-RC one). The claim on its evidence packet:

```
nominal: the state  $|x[k]|$  stays  $\leq 1.0$  for every step  $k$ 
faulted: inject a full-scale actuator fault,  $|x[k]|$  stays  $\leq 2.0$  for every step  $k$ 
```

Both held on real silicon. The question: how do you prove something about *every* step of an endless loop? You can't run it forever and check.

2. The one new idea: an "energy" that can only go downhill

First, vocabulary for a system that evolves in steps:

- **State** $x[k]$ — the one number (or vector) that captures "where the system is" at step k (here, the plant's output).
- **Dynamics** — the rule for the next step: $x[k+1] = f(x[k])$ (plant + controller together).
- **Invariant set** — a region such that *once the state is inside, it never leaves*. "Safe forever" = "the safe region is invariant."

The trick for proving invariance without simulating forever is a **Lyapunov function**: a kind of "energy" $V(x)$ (think $V(x) = x^2$ — distance from the target, squared) with one property:

every step, the energy does not increase: $V(x[k+1]) \leq V(x[k])$.

If energy can only stay flat or drop, the state can never spiral outward — so it's trapped in whatever region it started in. That's the whole idea: turn "for all infinitely many future steps" into "check one inequality about a single step." One local check $\Rightarrow$ a forever guarantee. (It's the continuous cousin of induction from Rung 1.5: prove the step, get all steps.)

3. Do it by hand

The kernels use the **contraction** version, which is clean enough to bound by hand. Suppose the plant's natural decay satisfies $|f(x)| \leq L \cdot |x| + c$ with $L < 1$ (it shrinks toward the origin each step, up to an offset c), and the controller can push by at most U and any fault by at most W . Then each step:

$$|x[k+1]| \leq L \cdot |x[k]| + (c + U + W)$$

Unrolling that (it's a geometric series — $L < 1$ is what makes it converge) gives a ceiling the state can never exceed:

$$|x[k]| \leq (c + U + W) / (1 - L) \quad \text{for all } k$$

That's the envelope. Plug in the nominal numbers and you get ≤ 1.0 ; add the actuator fault W and it grows to ≤ 2.0 — **exactly the two ceilings on the evidence packet**. The $L < 1$ is load-bearing: if the plant weren't a contraction, the series blows up and there's no envelope — which is why the nonlinear-plant lesson *measured* L first and honestly stopped if $L \geq 1$.

4. Make it formal

machlib proves the envelope as a theorem about the recurrence, `sorryAx`-free:

```
MachLib.Real.first_order_clamp_envelope    -- linear plant:   |x[k]| ≤ 1 (nominal), ≤ 1+Δ (fault)
MachLib.Real.nonlinear_drift_clamp_safe    -- |f(x)| ≤ L|x|+c, L<1 ⇒ |x| ≤ (c+U+W)/(1-L)
```

The proof is an induction (Rung 1.5) with the Lyapunov/contraction inequality as the step. Note what makes it robust: the bound holds for *any* controller output within the clamp — so it's immune to the Q16.16 quantization error you bounded on Rungs 1–3. Two layers of guarantee, stacked: the arithmetic is certified *and* the closed loop is certified, and neither subsumes the other.

5. Watch it hold

On silicon: the nominal run peaked at $\max |x| \approx 0.65 \leq 1.0$; with a full-scale fault injected, it peaked at $\approx 1.00 \leq 2.0$ — the state visibly rose above the nominal ceiling (the fault is real) but stayed under the proven fault ceiling. The envelope you derived, measured on real hardware, holding under a fault. That's the "guard you can feel."

The cliffhanger → Rung 6

You can now certify a number, a composition, and a loop-forever. Natural final question: **is there a limit?** What functions and systems can even *be* certified — is it everything, or is there a boundary where "provable" runs out? That question is real research, and it's the summit.

→ **Next:** Rung 6 — *What can even be certified?* — o-minimality, Pfaffian functions, and why `sin` and `exp` are "tame" enough to bound at all.

Exercises & checkpoint

1. With $|x[k+1]| \leq 0.9 \cdot |x[k]| + 0.1$ and $x[0] = 0$, compute $x[1]$, $x[2]$, $x[3]$, $x[4]$. What value is it creeping toward? Check it against the formula $(c)/(1-L) = 0.1/(1-0.9)$.
2. What goes wrong in that geometric series if $L \geq 1$? Why is $L < 1$ load-bearing for the whole envelope?
3. In one sentence: what is an *invariant set*, and why is "safe forever" the same as "the safe region is invariant"?

Checkpoint: you can derive the $(c+u+w)/(1-L)$ envelope and explain why $L < 1$ is required.